

Q1. Smart Hospital Patient Registration System**CODE**

```
import java.util.*;

public class Record {

    static class Patient {
        String id, name, mobile, department, doctor;

        Patient(String id, String name, String mobile, String department, String doctor) {
            this.id = id;
            this.name = name;
            this.mobile = mobile;
            this.department = department;
            this.doctor = doctor;
        }
    }

    static ArrayList<Patient> patients = new ArrayList<>();
    static HashMap<String, Patient> patientMap = new HashMap<>();

    static String properCase(String s) {
        s = s.trim().replaceAll("\\s+", " ");
        String[] words = s.split(" ");
        StringBuilder sb = new StringBuilder();

        for (String word : words) {
            if (word.contains(".")) {
                String[] parts = word.split("\\.");
                for (int i = 0; i < parts.length; i++) {
                    if (parts[i].length() > 0) {
                        sb.append(Character.toUpperCase(parts[i].charAt(0)));
                        if (parts[i].length() > 1)
                            sb.append(parts[i].substring(1).toLowerCase());
                    }
                    if (i != parts.length - 1)
                        sb.append(".");
                }
            } else {
                sb.append(Character.toUpperCase(word.charAt(0)));
                if (word.length() > 1)
                    sb.append(word.substring(1).toLowerCase());
            }
            sb.append(" ");
        }
        return sb.toString().trim();
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {

            int choice = Integer.parseInt(sc.nextLine());

            switch (choice) {

                case 1:

                    String id = sc.nextLine().trim();

                    if (patientMap.containsKey(id)) {
                        System.out.println("Duplicate Patient ID");

                        break;
                    }
                }
            }
        }
    }
}
```

```

    }

    String name = properCase(sc.nextLine());

    if (!name.matches("[A-Za-z ]+")) {
System.out.println("Invalid Patient Name");
break;
    }

    String mobile = sc.nextLine().trim();

    if (!mobile.matches("\\d{10}")) {
        System.out.println("Invalid Mobile Number");
break;
    }

    String department = sc.nextLine().trim().toUpperCase();

    String doctor = properCase(sc.nextLine());

    Patient p = new Patient(id, name, mobile, department, doctor);

    patients.add(p);
patientMap.put(id, p);

    System.out.println("Patient Registered Successfully.");

    break;

case 2:

    String searchId = sc.nextLine().trim();

    if (patientMap.containsKey(searchId)) {

        Patient pt = patientMap.get(searchId);

        System.out.println("Patient Details");
        System.out.println();
        System.out.println("ID : " + pt.id);
        System.out.println();
        System.out.println("Name : " + pt.name);
        System.out.println();
        System.out.println("Mobile : " + pt.mobile);
        System.out.println();
        System.out.println("Department : " + pt.department);
        System.out.println();
        System.out.println("Doctor : " + pt.doctor);

    } else {
        System.out.println("Patient Not Found");
    }

    break;

case 3:

    System.out.println("-----");
    System.out.println("ID Name Department");
    System.out.println("-----");

    for (Patient pt : patients) {
        System.out.println(pt.id + " " + pt.name + " " +
pt.department);
    }

```

```
        break;

    case 4:

        System.out.println("Department Count");
        System.out.println();

        HashMap<String, Integer> count = new HashMap<>();

        for (Patient pt : patients) {
            count.put(pt.department, count.getOrDefault(pt.department, 0) + 1);
        }

        for (Map.Entry<String, Integer> entry : count.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }

        break;

    case 5:
        return;
    }
}
}
```

INPUT

```
1
P101
Rahul Kumar
9876543210
Cardiology
Dr.Arun
1
P102
Priya
9876543211
Ortho
Dr.Meena
3
4
2
P101
5
```

OUTPUT

(no output)

Q2. Online Library Book Management System**CODE**

```
import java.util.*;

public class Record {

    static ArrayList<Book> bookList = new ArrayList<>();
    static HashMap<String, Book> bookMap = new HashMap<>();
    static LinkedHashSet<String> uniqueAuthors = new LinkedHashSet<>();
    static final int MAX_BOOKS = 500;
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int choice;

        do {
            choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1:
                    addBook();
                    break;
                case 2:
                    searchBook();
                    break;
                case 3:
                    displayAllBooks();
                    break;
                case 4:
                    displayUniqueAuthors();
                    break;
                case 5:
                    categoryWiseCount();
                    break;
                case 6:
                    break;
                default:
                    System.out.println("Invalid Choice");
            }

        } while (choice != 6);

        sc.close();
    }

    static void addBook() {

        if (bookList.size() >= MAX_BOOKS) {
            System.out.println("Maximum Book Limit Reached");
            return;
        }

        String bookId = sc.nextLine().trim().toUpperCase();

        if (bookMap.containsKey(bookId)) {
            sc.nextLine();
            sc.nextLine();
            sc.nextLine();
            sc.nextLine();
            System.out.println("Duplicate Book ID");
            return;
        }

        String title = toTitleCase(sc.nextLine().trim());

        if (title.isEmpty()) {
```

```

        System.out.println("Title cannot be
empty");
        return;
    }

    String author = toTitleCase(sc.nextLine().trim());

    if (author.isEmpty()) {
        System.out.println("Author cannot be
empty");
        return;
    }

    String category = sc.nextLine().trim().toUpperCase();

    if (category.isEmpty()) {
        System.out.println("Category cannot be
empty");
        return;
    }

    String publisher = toTitleCase(sc.nextLine().trim());

    if (publisher.isEmpty()) {
        System.out.println("Publisher name cannot be
empty");
        return;
    }

    Book book = new Book(bookId, title, author, category, publisher);

    bookList.add(book);
    bookMap.put(bookId, book);
    uniqueAuthors.add(author);

    System.out.println("Book Added Successfully.");
}

static void searchBook() {

    String bookId = sc.nextLine().trim().toUpperCase();

    if (bookMap.containsKey(bookId)) {

        Book book = bookMap.get(bookId);

        System.out.println("Book Details");
        System.out.println("Book ID : " + book.getBookId());
        System.out.println("Title : " + book.getTitle());
        System.out.println("Author : " + book.getAuthor());
        System.out.println("Category : " + book.getCategory());
        System.out.println("Publisher : " + book.getPublisher());

        System.out.println();

        stringOperations(book);

    } else {
        System.out.println("Book Not Available");
    }
}

static void displayAllBooks() {

    if (bookList.isEmpty()) {
        System.out.println("No Books Available");
        return;
    }

    System.out.println("-----");
    System.out.println("Book ID Title Author");
    System.out.println("-----");
}

```

```
        for (Book b : bookList) {
            System.out.println(
                b.getBookId() + " " +
                b.getTitle() + " " +
                b.getAuthor()
            );
        }
    }
    static void
displayUniqueAuthors() {

        if (uniqueAuthors.isEmpty()) {
            System.out.println("No Authors
Available");
            return;
        }

        System.out.println("Unique Authors");

        for (String author : uniqueAuthors) {
            System.out.println(author);
        }
    }

    static void categoryWiseCount() {

        LinkedHashMap<String, Integer> categoryCount = new LinkedHashMap<>();

        for (Book b : bookList) {

String category = b.getCategory();

            categoryCount.put(category,
categoryCount.getOrDefault(category, 0) + 1);
        }

        if (categoryCount.isEmpty()) {
            System.out.println("No Books
Available");
            return;
        }

        System.out.println("Category-wise Count");

        for (Map.Entry<String, Integer> entry : categoryCount.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }

        System.out.println();
    }

    static void stringOperations(Book b) {

String title = b.getTitle();

        System.out.println("Length : " + title.length());
        System.out.println("Words : " + title.trim().split("\\s+").length);
        System.out.println("Category : " + b.getCategory());
    }

    static String toTitleCase(String input) {
```

```
        input = input.trim();

        if (input.isEmpty())
            return "";

        String[] words = input.toLowerCase().split("\\s+");

        StringBuilder sb = new StringBuilder();

        for (String word : words) {

            sb.append(Character.toUpperCase(word.charAt(0)));

            if (word.length() > 1) {
                sb.append(word.substring(1));
            }

            sb.append(" ");
        }

        return sb.toString().trim();
    }
}

class Book {

    private String bookId;
    private String title;
    private String author;
    private String category;
    private String publisher;

    public Book(String bookId,
                String title,
                String author,
                String category,
                String publisher) {

        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.category = category;
        this.publisher = publisher;
    }

    public String getBookId() {
        return bookId;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getCategory() {
        return category;
    }

    public String getPublisher() {
        return publisher;
    }
}
```

INPUT

```
1
B101
Java Programming
Herbert Schildt
Programming
McGraw Hill
1
B102
Data Structures
Mark Allen
Computer Science
Pearson
3
4
5
2
B101
6
```

OUTPUT

(no output)

